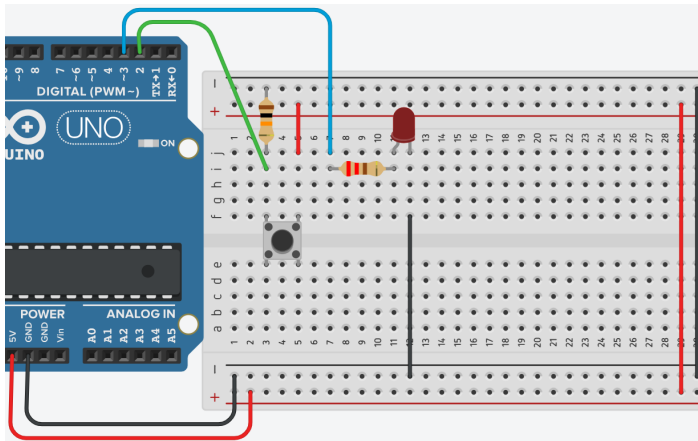


LED con Pulsador - IF



Explicación del código

Este programa implementa un interruptor tipo "toggle" usando un pulsador y un LED. Cada vez que se presiona el botón, el LED cambia de estado (se enciende si estaba apagado, o se apaga si estaba encendido). Es una introducción al manejo de entradas digitales, variables booleanas y la estructura condicional `if`.

1. Declaración de variables globales

```
int btn_e = 2;
int led_s = 3;
bool estado = LOW;
```

- `int btn_e = 2;`: Define el pin digital 2 como entrada para el pulsador.
- `int led_s = 3;`: Define el pin digital 3 como salida para el LED. Este pin tiene capacidad PWM, aunque en este programa solo se usa `digitalWrite()`.
- `bool estado = LOW;`: Declara una variable de tipo `bool` (booleana) llamada `estado` que almacena el estado actual del LED (apagado o encendido). Se inicializa en `LOW` (apagado). Una variable `bool` solo puede tener dos valores: `true` (1) o `false` (0), que en Arduino equivalen a `HIGH` y `LOW`.

2. Configuración `setup()`

```
void setup()
{
  pinMode(led_s, OUTPUT);
  pinMode(btn_e, INPUT);
}
```

- `pinMode(led_s, OUTPUT);`: Configura el pin del LED como salida digital.
- `pinMode(btn_e, INPUT);`: Configura el pin del pulsador como entrada. **Importante:** Para que la lectura sea estable, se debe conectar una resistencia externa de pull-down (10 kΩ a tierra) o usar la resistencia pull-up interna con `INPUT_PULLUP`. El código original asume una conexión con pull-down externo.

3. Bucle `loop()` con estructura `if`

```

void loop()
{
    //Presiono el botón y enciende led, vuelvo a
    //presionar el botón y se apaga el led
    // declara "bool estado = LOW;" arriba
    if (digitalRead(btn_e) == HIGH)
    {
        estado = !estado;
    }
    digitalWrite(led_s, estado);
    //delay(1000);
}

```

Lectura del pulsador y cambio de estado

- `if (digitalRead(btn_e) == HIGH)`: Comprueba si el pulsador está presionado (valor `HIGH` en el pin). Si es cierto, ejecuta el bloque interior.
- `estado = !estado;`: Esta línea invierte el valor de la variable `estado`. El operador `!` es la negación lógica: si `estado` era `LOW` (apagado), pasa a `HIGH` (encendido); si era `HIGH`, pasa a `LOW`. Cada presión del botón cambia el estado del LED.

Escritura en el LED

- `digitalWrite(led_s, estado);`: Escribe en el pin del LED el valor actual de `estado`. Si `estado` es `HIGH`, el LED se enciende; si es `LOW`, se apaga.

Problema del rebote (bouncing)

El código actual tiene una limitación: los pulsadores mecánicos producen múltiples transiciones `HIGH/LOW` en pocos milisegundos (rebote). Esto puede causar que el LED cambie de estado varias veces con una sola presión. El `delay(1000);` está comentado, por lo que no hay ningún antirrebote. Una solución simple es añadir un pequeño `delay(50);` después de `estado = !estado;` para ignorar las lecturas inestables.

Código completo para copiar y pegar

```

// LED con Pulsador - IF (Toggle)
// Conexiones:
// - LED ánodo → pin 3 (con resistencia 220Ω a GND)
// - Pulsador → pin 2 y GND (con resistencia pull-down de 10kΩ)

int btn_e = 2;
int led_s = 3;
bool estado = LOW;

void setup()
{
    pinMode(led_s, OUTPUT);
}

```

```
pinMode(btn_e, INPUT);
}

void loop()
{
  if (digitalRead(btn_e) == HIGH)
  {
    estado = !estado;
    delay(50); // Pequeña pausa para antirrebote (opcional)
  }
  digitalWrite(led_s, estado);
}
```

Enlace al simulador

[Código en Tinkercad](#)

Preguntas teóricas

1. ¿Qué es una variable de tipo `bool`? ¿Qué valores puede almacenar y para qué se utiliza en este programa?
2. Explica el significado de la línea `estado = !estado;` y cómo logra cambiar el estado del LED cada vez que se presiona el pulsador. ¿Qué otro operador lógico podría usarse para el mismo fin?
3. ¿Qué es el "rebote" (bouncing) de un pulsador y por qué puede afectar al funcionamiento de este programa? ¿Cómo se podría solucionar?
4. En el código, la línea `delay(1000);` está comentada. ¿Qué efecto tendría si se activara? ¿Por qué el programador decidió comentarla?
5. ¿Por qué es necesario configurar el pin del pulsador como `INPUT`? ¿Qué sucede si se configura como `OUTPUT` por error? Analiza las consecuencias.

Ejercicios prácticos (modificar el código y anotar cambios)

Instrucciones: Para cada ejercicio, copia el código original, realiza la modificación indicada, carga el programa en el simulador (o en el Arduino real) y describe cómo cambia el comportamiento del circuito.

Ejercicio 1

Implementa una rutina de antirrebote simple añadiendo un `delay(50);` justo después de `estado = !estado;`. Observa la diferencia.

Pregunta: ¿El comportamiento mejoró? ¿Qué sucede si aumentas el delay a 200 ms? ¿Y si lo reduces a 5 ms?

Ejercicio 2

Modifica el programa para que el LED parpadee 3 veces rápidamente cada vez que se presiona el botón, en lugar de simplemente cambiar su estado. Utiliza un bucle `for`.

Pregunta: ¿Cómo se comporta ahora el circuito? ¿El parpadeo es siempre el mismo independientemente de cuánto tiempo se mantenga presionado el botón?

Ejercicio 3

Cambia la lógica del pulsador para que use una resistencia pull-up interna. Para ello, configura `pinMode(btn_e, INPUT_PULLUP);` y modifica la condición del `if` para que detecte `LOW` en lugar de `HIGH`.

Pregunta: ¿Necesitas cambiar el conexionado del pulsador? ¿Qué ventaja tiene usar la resistencia pull-up interna?

Ejercicio 4

Añade un segundo LED en el pin 4. Modifica el programa para que el segundo LED siga el estado invertido del primero: cuando el primer LED esté encendido, el segundo debe estar apagado, y viceversa.

Pregunta: ¿Qué observas? ¿Cómo se logra la inversión?

Ejercicio 5

Reemplaza el LED por un servomotor (conectado al pin 9). Modifica el código para que cada vez que se presione el botón, el servomotor cambie entre dos posiciones (por ejemplo, 0° y 90°).

Pregunta: ¿El comportamiento es similar al del LED? ¿Qué librería necesitas incluir y cómo se controla el servomotor?

Entregar las respuestas a las preguntas teóricas y la descripción de los cambios observados en cada ejercicio.